

Rapport de Projet : FlipTek

Projet FlipTek - Filière préING1 - 2025-2026

Groupe MEF 2-C : Cyprien Bayard, Mathis Gaston-Breton, Titouan Le Mirronet

1. Présentation de l'équipe et choix du projet

Nous sommes Cyprien Bayard, Mathis Gaston-Breton et Titouan Le Mirronet, et nous formons le groupe MEF 2-C. Pour ce projet, nous avons choisi de développer une version terminale du jeu de cartes Flip 7 : rebaptisé FlipTek pour l'occasion. Nous avons codé ce jeu en C en environnement Linux

Le choix s'est fait assez naturellement : nous connaissions tous les trois ce jeu et l'apprécions déjà. L'idée de le coder nous semblait donc à la fois motivante et cohérente, car nous partions avec une bonne compréhension des règles et une envie réelle de voir tourner le résultat. C'est ce genre de projet où on a envie d'ouvrir son terminal pour tester, pas seulement pour vérifier que ça compile.

2. Organisation et flux de travail

Dès le départ, on a essayé de poser les bases sérieusement plutôt que de se lancer tête baissée dans le code. On s'est retrouvés une première fois pour lire la consigne ensemble, comprendre les règles du jeu dans le détail, et se répartir le travail.

Pour la communication au quotidien, on a utilisé un groupe WhatsApp, ce qui nous permettait de signaler rapidement une avancée, un bug ou une question sans avoir besoin de se retrouver physiquement. Pour le code, on a mis en place un dépôt GitHub commun dès le début, ce qui nous a évité les échanges de fichiers par messages et les versions qui se perdent. Les moments les plus productifs ont clairement été les sessions en salle informatique. Une fois réunis devant les écrans, on avançait vite : on s'expliquait ce qu'on avait fait chacun de son côté, on intégrait les morceaux ensemble, on déboguait en direct. Il y a quelque chose d'efficace dans le fait de pouvoir pointer un bout de code à quelqu'un et lui dire "regarde, ça fait n'importe quoi ici", c'est bien plus rapide qu'un message écrit.

Une dernière session collective a permis de tout finaliser et de stabiliser le programme avant le rendu.

3. Répartition des tâches

On a décidé de se répartir les fonctions à implémenter plutôt que de diviser le projet en grandes zones étanches. Concrètement :

- Cyprien s'est occupé principalement des fonctions d'affichage : la mise en forme du terminal, la façon dont les cartes et l'état du jeu sont présentés à l'écran. Ainsi que plusieurs fonctions.

- Titouan a pris en charge le cœur d'une manche : la boucle de jeu principale, la gestion des tours de table, les conditions d'arrêt, et les fonctions internes à cette logique.
- Mathis a contribué à plusieurs fonctions communes, en particulier autour de la gestion des scores et de certaines parties de la logique de jeu.

En pratique, les frontières n'étaient pas rigides : chacun a aussi mis la main sur des fonctions qui dépassaient son domaine principal, surtout lors des sessions collectives. C'est d'ailleurs souvent comme ça qu'on a détecté et corrigé le plus de problèmes.

4. Approche technique et choix de développement

4.1 Développer par étapes

Au tout début, on a eu le réflexe classique : essayer de tout coder d'un coup, en imaginant directement le programme final avec plusieurs joueurs, plusieurs manches, les scores, etc. Résultat : on s'est retrouvés avec un code difficile à tester et à déboguer.

Sur les conseils de notre chargé de TD, on a changé d'approche et adopté une méthode incrémentale : d'abord un joueur et une manche, puis plusieurs joueurs sur une manche, puis plusieurs manches formant une partie complète, et enfin l'ajout du calcul et de l'affichage des scores. Cette façon de faire a vraiment changé notre manière de travailler, valider chaque étape avant de passer à la suivante nous a permis d'avancer beaucoup plus sereinement.

4.2 Le casse-tête de la pioche

La difficulté technique la plus marquante du projet a été la gestion de la pioche. Le problème : les cartes doivent défiler dans un ordre fixé dès le début de la partie, sans jamais être supprimées définitivement du tableau elles doivent simplement ne plus être accessibles une fois tirées.

On a finalement opté pour une solution élégante dans sa simplicité : un tableau de 85 cartes mélangé aléatoirement au lancement, parcouru grâce à un indice courant. La fonction "carte du dessus" lit la carte à cet indice, puis l'incrémente pour avancer dans la pioche. Pas de suppression, pas de déplacement de cases : juste un pointeur qui avance. Ça peut sembler évident une fois trouvé, mais on a tâtonné un moment avant d'arriver à cette solution.

4.3 Robustesse des saisies

Un autre défi que l'on n'avait pas anticipé : la robustesse face aux saisies utilisateur. Le programme fonctionnait bien pour des entrées correctes, mais dès qu'on tapait deux lettres d'un coup ou qu'on appuyait plusieurs fois sur Entrée, il affichait plusieurs fois le même message ou se comportait de façon bizarre sans planter, mais sans se comporter correctement non plus.

On a dû apprendre à bien utiliser le buffer, ce qui nous a aidé à résoudre plusieurs problèmes de compilations.

4.4 Autres choix

Nous avons choisi d'écrire l'ensemble du code en français (noms de variables, fonctions, commentaires). La consigne l'autorisait, et nous avons estimé que cela faciliterait la relecture et la communication au sein du groupe. Le code est également structuré en plusieurs fichiers distincts, conformément aux critères généraux du projet.

5. Les améliorations qu'on n'a pas faites, et pourquoi

Honnêtement, les améliorations optionnelles nous intéressaient. L'idée de jouer contre un ordinateur, ajouter des cartes spéciales ou une interface graphique était séduisante, surtout parce qu'on avait pris du plaisir sur le projet et qu'on aurait bien continué à le faire évoluer.

Mais on a fait le choix de ne pas les implémenter, pour une raison simple : on était à un stade avancé du projet, et on ne voulait pas prendre le risque de déstabiliser un code qui fonctionnait bien. Ajouter des fonctionnalités complexes en fin de développement, c'est souvent là qu'on introduit des bugs difficiles à corriger. On a préféré livrer quelque chose de stable et complet plutôt que de viser plus haut et risquer de tout casser. Nous n'avons donc pas implémenté les cartes spéciales STOP, 2ème chance et 3 à la suite, la partie personnalisée, le mode joueur contre ordinateur, ni l'interface graphique.

6. Résultat et bilan

On est assez fiers du résultat. Ce qui nous a le plus surpris, c'est que le jeu rend vraiment bien dans le terminal. On aurait pu penser qu'un jeu de cartes sans interface graphique serait austère ou peu lisible, mais la simplicité du Flip 7 c'est fondamentalement juste des cartes et des chiffres se prête bien à ce type d'affichage. Avec un peu de soin sur la mise en forme, le jeu est agréable à utiliser.

Toutes les fonctionnalités obligatoires sont présentes : gestion de n'importe quel nombre de joueurs, pioche de 85 cartes, règles complètes (doublon, 7 cartes différentes, bonus de 15 points, cartes bonus), fin de partie à 200 points ou pioche épuisée, affichage des statistiques de pioche pendant le jeu, et sauvegarde des scores dans un fichier texte.

Sur le plan humain, le projet s'est très bien passé. Pas de friction au sein du groupe, une bonne répartition du travail, et une vraie motivation partagée. C'est peut-être ce qui explique qu'on soit arrivés au bout sans trop de douleur.